



Certified Kubernetes Administrator (CKA)

By Cloud Mentor Pro

Updated: 10/04/2026



Table of Contents

- Section 21 - Mock Exam - 1
 - *Mock Exam - 1*
 - *Mock Exam - 2 (Level Up!)*
 - *Mock Exam - 3 (Wohooo! Level Up!)*

Mock Exam - 1

- *12 Questions*

QUESTION 1 OF 12

Weight: 8

Create a Pod `mc-pod` in the `mc-namespace` namespace with three containers. The first container should be named `mc-pod-1`, run the `nginx:1-alpine` image, and set an environment variable `NODE_NAME` to the node name. The second container should be named `mc-pod-2`, run the `busybox:1` image, and continuously log the output of the `date` command to the file `/var/log/shared/date.log` every second. The third container should have the name `mc-pod-3`, run the image `busybox:1`, and print the contents of the `date.log` file generated by the second container to stdout. Use a shared, non-persistent volume.

- ☐ Is the pod environment variable correctly set?
- ☐ Does the Sidecar display logs?

Solution

Ensure the shared volume is not persisted across restarts.
Here's the YAML for the Pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: mc-pod
  namespace: mc-namespace
spec:
  containers:
    - name: mc-pod-1
      image: nginx:1-alpine
      env:
        - name: NODE_NAME
          valueFrom:
            fieldRef:
              fieldPath: spec.nodeName
    - name: mc-pod-2
      image: busybox:1
      volumeMounts:
        - name: shared-volume
          mountPath: /var/log/shared
      command:
        - "sh"
        - "-c"
        - "while true; do date >> /var/log/shared/date.log; sleep 1; done"
    - name: mc-pod-3
      image: busybox:1
      command:
        - "sh"
        - "-c"
        - "tail -f /var/log/shared/date.log"
      volumeMounts:
        - name: shared-volume
          mountPath: /var/log/shared
  volumes:
    - name: shared-volume
      emptyDir: {}
```

QUESTION 2 OF 12

Weight: 7

This question needs to be solved on node `node01`. To access the node using SSH, use the credentials below:

```
username: bob
password: caleston123
```

As an administrator, you need to prepare `node01` to install kubernetes. One of the steps is installing a container runtime. Install the `cri-docker_0.3.16.3-0.debian.deb` package located in `/root` and ensure that the `cri-docker` service is running and enabled to start on boot.

- ☐ Is the package installed successfully on node01?
- ☐ Is the service running?
- ☐ Is the service enabled?

Solution

Use `dpkg` to install the package and `systemctl` to manage the service. SSH to node01 as follows, using password `caleston123`:

```
ssh bob@node01
```

Switch to root using `sudo -i` or prefix the commands below using `sudo`.

The `cri-docker_0.3.16.3-0.debian.deb` package is located in the `/root` directory. Use `dpkg` to install the package:

```
dpkg -i /root/cri-docker_0.3.16.3-0.debian.deb
```

After installing the package, start the `cri-docker` service and enable it to start on boot:

```
systemctl start cri-docker
systemctl enable cri-docker
```

Verify that the `cri-docker` service is running:

```
systemctl is-active cri-docker
```

Check that it is enabled to start on boot:

```
systemctl is-enabled cri-docker
```

You should see `active` and `enabled` as the output for both commands

QUESTION 3 OF 12

Weight: 6

On `controlplane` node, identify all CRDs related to VerticalPodAutoscaler and save their names into the file `/root/vpa-crds.txt`.

☐ Does the file contain correct CRDs?

Solution

Use `kubectl get crd` to find all CRDs and filter by `VerticalPodAutoscaler`.
Remember to exit back to `controlplane` node before attempting this question.

To find the CRDs Related to VerticalPodAutoscaler, use the following `kubectl` command to list all CRDs and filter them for VerticalPodAutoscaler resources:

```
kubectl get crd -o custom-columns=NAME:.metadata.name | grep verticalpodautoscaler > /root/vpa-crds.txt
```

Ensure that the CRD names are correctly saved in the `/root/vpa-crds.txt`.
The file should contain the CRD names related to VerticalPodAutoscaler.

QUESTION 4 OF 12

Weight: 8

Create a service named `messaging-service` to expose the `messaging` pod within the cluster on port `6379`. The `messaging` pod is running in the `default` namespace.

Use imperative commands.

- ☐ Service: messaging-service
- ☐ Port: 6379
- ☐ Type: ClusterIp
- ☐ Use the right labels

Solution

Run the command: `kubectl expose pod messaging --port=6379 --name messaging-service`

QUESTION 5 OF 12

Weight: 10

Create a deployment named `hr-web-app` using the image `kodekloud/webapp-color` with `2` replicas.

- ☐ Name: hr-web-app
- ☐ Image: kodekloud/webapp-color
- ☐ Replicas: 2

Solution

Run the command: `kubect1 create deployment hr-web-app --image=kodekloud/webapp-color --replicas=2`

QUESTION 6 OF 12

Weight: 12

A new application `orange` is deployed. There is something wrong with it. Identify and fix the issue.

☐ Is the issue fixed?

Solution

To know more details of `orange` pod:

```
kubectl describe po orange
```

and look under the `initContainers` section. There is an issue with the given command.

```
Command:
  sh
  -c
  sleeep 2;
```

In the above, we need to correct the `sleeep` command.

To update the pod with an easiest way by running command:

```
kubectl edit po orange
```

It's not possible to update the changes in the running pod so after saving the changes. It will create a temporary file in the default location `/tmp/`.

Use that manifest file and replace with the existing pod:

```
kubectl replace -f /tmp/kubectl-edit-xxxx.yaml --force
```

Above command will delete the existing pod and will recreate the new pod with latest changes.

QUESTION 7 OF 12

Weight: 8

Expose the `hr-web-app` created in the previous task as a service named `hr-web-app-service`, accessible on port `30082` on the nodes of the cluster.

The web application listens on port `8080`.

- ☐ Name: hr-web-app-service
- ☐ Type: NodePort
- ☐ Endpoints: 2
- ☐ Port: 8080
- ☐ NodePort: 30082

Solution

Run the command: `kubect1 expose deployment hr-web-app --type=NodePort --port=8080 --name=hr-web-app-service --dry-run=client -o yaml > hr-web-app-service.yaml` to generate a service definition file.

Now, in generated service definition file add the `nodePort` field with the given port number under the `ports` section and create a service.

QUESTION 8 OF 12

Weight: 8

Create a `Persistent Volume` with the given specification: -

Volume name: `pv-analytics`

Storage: `100Mi`

Access mode: `ReadWriteMany`

Host path: `/pv/data-analytics`

- ☐ Is the volume name set?
- ☐ Is the storage capacity set?
- ☐ Is the accessMode set?
- ☐ Is the hostPath set?

Solution

Solution manifest file to create a persistent volume `pv-analytics` as follows:

```
---
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-analytics
spec:
  capacity:
    storage: 100Mi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteMany
  hostPath:
    path: /pv/data-analytics
```

QUESTION 9 OF 12

Weight: 10

Create a Horizontal Pod Autoscaler (HPA) with name `webapp-hpa` for the deployment named `kkapp-deploy` in the **default namespace** with the `webapp-hpa.yaml` file located under the root folder.

Ensure that the HPA scales the deployment based on **CPU utilization**, maintaining an average CPU usage of **50%** across all pods.

Configure the HPA to **cautiously scale down** pods by setting a **stabilization window of 300 seconds** to prevent rapid fluctuations in pod count.

Note: The `kkapp-deploy` deployment is created for backend; you can check in the terminal.

- ☐ Is the HPA `webapp-hpa` deployed?
- ☐ Is the deployment configured for metrics CPU Utilization?
- ☐ Is the stabilization window set to 300 seconds?

Solution

Under `/root/` folder you will find a yaml file `webapp-hpa.yaml`. Update the yaml file as per task given.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: webapp-hpa
  namespace: default
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: kkapp-deploy
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
  behavior:
    scaledDown:
      stabilizationWindowSeconds: 300
```

Use below command

```
kubectl create -f webapp-hpa.yaml
```

QUESTION 10 OF 12

Weight: 9

Deploy a Vertical Pod Autoscaler (VPA) with name `analytics-vpa` for the deployment named `analytics-deployment` in the default namespace. The VPA should automatically adjust the CPU and memory requests of the pods to optimize resource utilization. Ensure that the VPA operates in `Recreate` mode, allowing it to evict and recreate pods with updated resource requests as needed.

- ☐ Is the VPA `analytics-vpa` created for deployment `analytics-deployment` ?
- ☐ Is the updatePolicy set to `Recreate` mode for deployment `analytics-deployment` ?

Solution

Use the below YAML file to create the VPA for deployment `analytics-deployment`

```
kubectl create -n default -f - <<EOF
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: analytics-vpa
  namespace: default
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: analytics-deployment
  updatePolicy:
    updateMode: "Recreate"
EOF
```

QUESTION 11 OF 12

Weight: 6

Create a **Kubernetes Gateway** resource with the following specifications:

1. Name: `web-gateway`
2. Namespace: `nginx-gateway`
3. Gateway Class Name: `nginx`
4. Listeners:
 - Protocol: `HTTP`
 - Port: `80`
 - Name: `http`

☐ Is the web-gateway deployed to listen on port 80?

Solution

Copy the below YAML file to the terminal and create a gateway resource.

```
kubectl create -n nginx-gateway -f - <<EOF
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: web-gateway
  namespace: nginx-gateway
spec:
  gatewayClassName: nginx
  listeners:
    - name: http
      protocol: HTTP
      port: 80
EOF
```

QUESTION 12 OF 12

Weight: 8

One co-worker deployed a `podinfo` helm chart `kk-mock1` in the `kk-ns` namespace on the `cluster`. A new update is pushed to the helm chart, and the team wants you to update the helm repository to fetch the new changes.

After updating the helm chart, upgrade the helm chart version to `6.11.2`.

- ☐ Is the deployment running?
- ☐ Is the chart version upgraded?

Solution

In this task, we will use the `kubectl` and `helm` commands. Here are the steps: -

1. use the `helm ls` command to list all the releases installed using Helm in the Kubernetes cluster.

```
helm ls -A
```

Here `-A` or `--all-namespaces` option lists all the releases of all the namespaces.

2. Identify the namespace where the resources get deployed.

3. Use the `helm repo ls` command to list the helm repositories.

```
helm repo ls
```

4. Now, update the helm repository with the following command: -

```
helm repo update kk-mock1 -n kk-ns
```

The above command updates the local cache of available charts from the configured chart repositories.

5. The `helm search` command searches for all the available charts in a specific Helm chart repository. In our case, it's the podinfo helm chart.

```
helm search repo kk-mock1/podinfo -n kk-ns -1 | head -n30
```

The `-1` or `--versions` option is used to display information about all available chart versions.

6. Upgrade the helm chart to `6.11.2`: -

```
helm upgrade kk-mock1 kk-mock1/podinfo -n kk-ns --version=6.11.2
```

7. After upgrading the chart version, you can verify it with the following command: -

```
helm ls -n kk-ns
```

Look under the `CHART` column for the chart version.